

Estruturas Espaciais

INF2604 – Geometria Computacional

Waldemar Celes
celes@inf.puc-rio.br

Departamento de Informática, PUC-Rio



Tópicos

Estruturas de Busca

Campo de Distância

Aritmética Robusta



Estruturas de Busca



Localização de pontos

Problema: Dado um ponto (x, y) verificar se ele já existe no conjunto



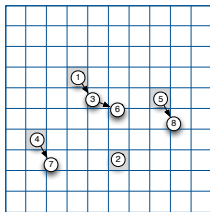
Localização de pontos

Problema: Dado um ponto (x, y) verificar se ele já existe no conjunto

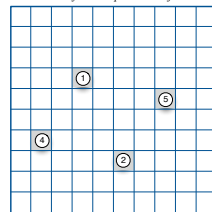
Grade regular como estrutura de aceleração

- ▶ Escolha uma resolução: Δ
- ▶ Crie a estrutura da grade
- ▶ Cada célula armazena a lista de pontos que contém

Grade com listas



Grade com referência para um objeto



Vetor auxiliar



Grade regular

Construção da grade regular

- Determina a caixa alinhada envolvente do conjunto de pontos

$$x_{min}, x_{max}, y_{min}, y_{max}$$

- Cria estrutura de grade: $N_x \times N_y$

$$N_x = \left\lceil \frac{x_{max} - x_{min}}{\Delta} \right\rceil, \quad N_y = \left\lceil \frac{y_{max} - y_{min}}{\Delta} \right\rceil$$

- Armazena pontos nas células da grade
 - Localiza célula que contém ponto

$$\langle i, j \rangle = \left\langle \left\lfloor \frac{x - x_{min}}{\Delta} \right\rfloor, \left\lfloor \frac{y - y_{min}}{\Delta} \right\rfloor \right\rangle$$

- Insere ponto na lista da célula



Grade regular

Algoritmo de **localização de ponto**

- ▶ Localiza célula que contém ponto

$$\langle i, j \rangle = \left\langle \left\lfloor \frac{x - x_{min}}{\Delta} \right\rfloor, \left\lfloor \frac{y - y_{min}}{\Delta} \right\rfloor \right\rangle$$

- ▶ Busca ponto na lista da célula



Grade regular

Algoritmo de **localização de ponto**

- ▶ Localiza célula que contém ponto

$$\langle i, j \rangle = \left\langle \left\lfloor \frac{x - x_{min}}{\Delta} \right\rfloor, \left\lfloor \frac{y - y_{min}}{\Delta} \right\rfloor \right\rangle$$

- ▶ Busca ponto na lista da célula

Tempo esperado para localização



Grade regular

Algoritmo de **localização de ponto**

- ▶ Localiza célula que contém ponto

$$\langle i, j \rangle = \left\langle \left\lfloor \frac{x - x_{min}}{\Delta} \right\rfloor, \left\lfloor \frac{y - y_{min}}{\Delta} \right\rfloor \right\rangle$$

- ▶ Busca ponto na lista da célula

Tempo esperado para localização

- ▶ $O(k)$, onde k é a número de pontos médio por célula



Grade regular

Algoritmo de **localização de ponto**

- ▶ Localiza célula que contém ponto

$$\langle i, j \rangle = \left\langle \left\lfloor \frac{x - x_{min}}{\Delta} \right\rfloor, \left\lfloor \frac{y - y_{min}}{\Delta} \right\rfloor \right\rangle$$

- ▶ Busca ponto na lista da célula

Tempo esperado para localização

- ▶ $O(k)$, onde k é a número de pontos médio por célula

Espaço de armazenamento



Grade regular

Algoritmo de **localização de ponto**

- ▶ Localiza célula que contém ponto

$$\langle i, j \rangle = \left\langle \left\lfloor \frac{x - x_{min}}{\Delta} \right\rfloor, \left\lfloor \frac{y - y_{min}}{\Delta} \right\rfloor \right\rangle$$

- ▶ Busca ponto na lista da célula

Tempo esperado para localização

- ▶ $O(k)$, onde k é a número de pontos médio por célula

Espaço de armazenamento

- ▶ $O(N_x N_y + n)$



Grade regular

Grade regular esparsa

- ▶ Estrutura original tem alto custo de memória

$$N_x N_y \gg n$$



Grade regular

Grade regular esparsa

- Estrutura original tem alto custo de memória

$$N_x N_y \gg n$$

- Alternativa: uso de tabela de dispersão (*hash*)

```
int hash (int i, int j, int k)
{
    const int h1 = 0x8da6b343;
    const int h2 = 0x8163841;
    const int h3 = 0xcb1ab31f;

    int n = (h1*i + h2*j + h3*k) % NBUCKETS;
    if (n < 0)
        n += NBUCKETS;
    return n;
}
```



Localização de pontos

Localização de pontos com tolerância r

$$\sqrt{(x - p_x)^2 + (y - p_y)^2} \leq r$$



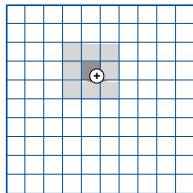
Localização de pontos

Localização de pontos com tolerância r

$$\sqrt{(x - p_x)^2 + (y - p_y)^2} \leq r$$

Algoritmo de localização

- ▶ Localiza célula que contém ponto
 - ▶ Busca também nas células vizinhas



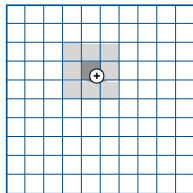
Localização de pontos

Localização de pontos com tolerância r

$$\sqrt{(x - p_x)^2 + (y - p_y)^2} \leq r$$

Algoritmo de localização

- ▶ Localiza célula que contém ponto
- ▶ Busca também nas células vizinhas



Condição para corretude: $\Delta \geq r$



Interseção entre discos

Condição de interseção entre discos

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \leq r_1 + r_2$$



Interseção entre discos

Condição de interseção entre discos

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \leq r_1 + r_2$$

Grade regular

- ▶ Estratégia 1: cada disco em uma célula
 - ▶ Centro do disco determina célula a que pertence
 - ▶ Algoritmo de localização visita células vizinhas
 - ▶ Condição para corretude: $\Delta \geq 2r_{max}$



Interseção entre discos

Grade regular

- ▶ Estratégia 2: cada disco em k células
 - ▶ Disco inserido na lista de cada célula com interseção



Interseção entre discos

Grade regular

- ▶ Estratégia 2: cada disco em k células
 - ▶ Disco inserido na lista de cada célula com interseção

Algoritmo de interseção

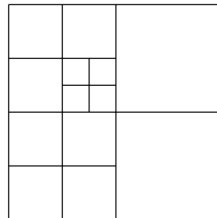
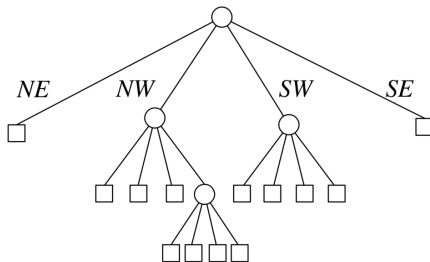
- ▶ Localiza células que têm interseção com disco
- ▶ Verifica interseção entre discos de cada célula
 - ▶ Evita duplicidade pela posição do ponto de interseção



Quadtree

Subdivisão adaptativa

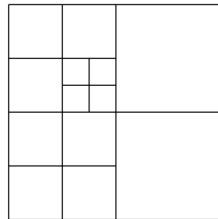
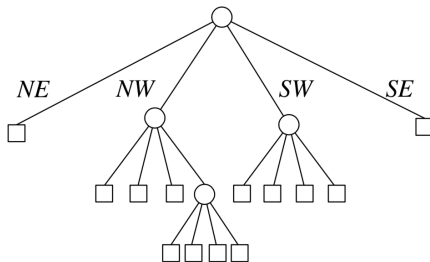
- ▶ Célula subdividida até que se alcance o número de objetos desejado
- ▶ Célula representa um nó da quadtree



Quadtree

Subdivisão adaptativa

- ▶ Célula subdividida até que se alcance o número de objetos desejado
- ▶ Célula representa um nó da quadtree



- ▶ Extensão natural para 3D: octree



Quadtree

Representação explícita

- ▶ Árvore de ponteiros



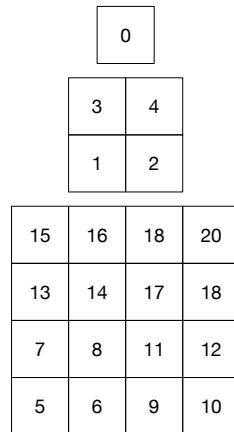
Quadtree

Representação explícita

- ▶ Árvore de ponteiros

Representação implícita

- ▶ Células armazenadas em vetor
- ▶ Identificação de nós sequencial



Quadtree

Representação explícita

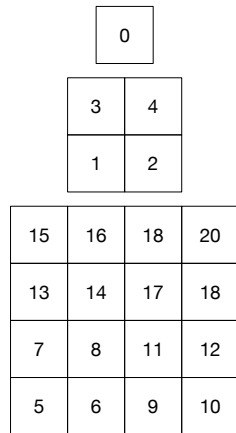
- ▶ Árvore de ponteiros

Representação implícita

- ▶ Células armazenadas em vetor
- ▶ Identificação de nós sequencial
- ▶ Acesso ao nó pai

$$p = \left\lfloor \frac{i - 1}{4} \right\rfloor$$

- ▶ Acesso ao filho k ($k = 1, 2, 3, 4$)
 $f = 4i + k$



Quadtree

Representação explícita

- ▶ Árvore de ponteiros

Representação implícita

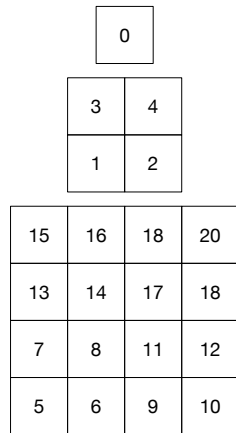
- ▶ Células armazenadas em vetor
- ▶ Identificação de nós sequencial
- ▶ Acesso ao nó pai

$$p = \left\lfloor \frac{i - 1}{4} \right\rfloor$$

- ▶ Acesso ao filho k ($k = 1, 2, 3, 4$)

$$f = 4i + k$$

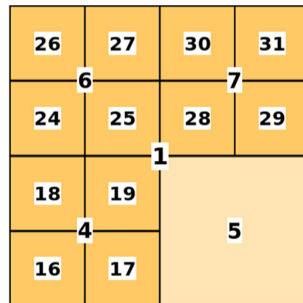
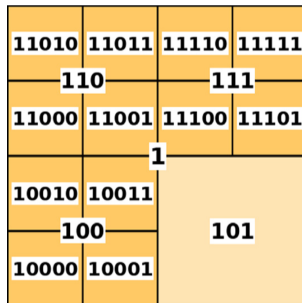
- ▶ Esparsidade contornada com uso de *hash*



Quadtree

Identificação dos nós baseada no código Morton

- Codificação binária com coerência espacial
- Acesso a nós adjacentes não irmãos



Kd-tree

Intercala direção de plano de corte

- ▶ Balanceamento adaptativo com base em algum critério
 - ▶ Por exemplo, mesmo número de objetos em cada partição

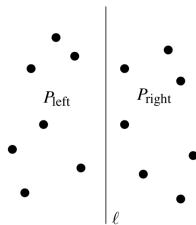


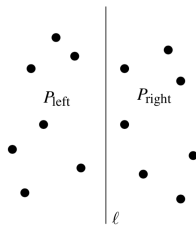
Figura extraída de Computational Geometry, de Berg et al., 2008



Kd-tree

Intercala direção de plano de corte

- ▶ Balanceamento adaptativo com base em algum critério
 - ▶ Por exemplo, mesmo número de objetos em cada partição



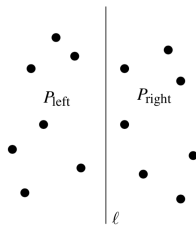
- ▶ Não necessariamente é uma árvore binária



Kd-tree

Intercala direção de plano de corte

- ▶ Balanceamento adaptativo com base em algum critério
 - ▶ Por exemplo, mesmo número de objetos em cada partição



- ▶ Não necessariamente é uma árvore binária
- ▶ Extensão direta para n -dimensão



Figura extraída de Computational Geometry, de Berg et al., 2008



Kd-tree

Aplicação em **buscas ortogonais**

- ▶ Exemplo: interpretação geométrica de seleção em base de dados
 - ▶ Intervalo: $[x, x'] \times [y, y']$
 - ▶ Resposta: $\{p_i / p_x \in [x, x'] \text{ e } p_y \in [y, y']\}$

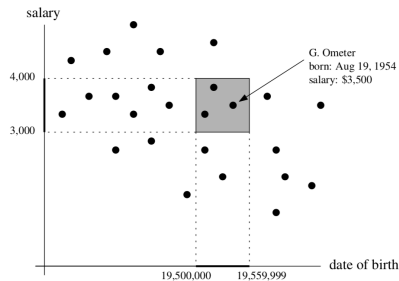


Figura extraída de Computational Geometry, de Berg et al., 2008



Kd-tree

Construção

- ▶ Partição à esquerda é intervalo fechado à direita
 - ▶ Ponto sobre plano divisor armazenado à esquerda
 - ▶ Assume, a princípio que não existem coordenadas x ou y coincidentes

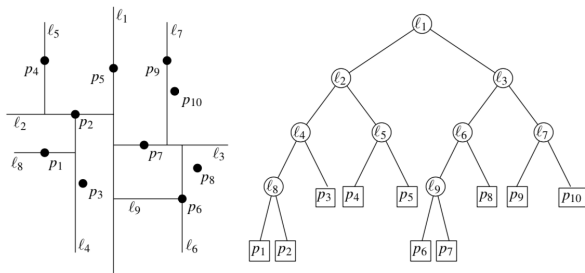
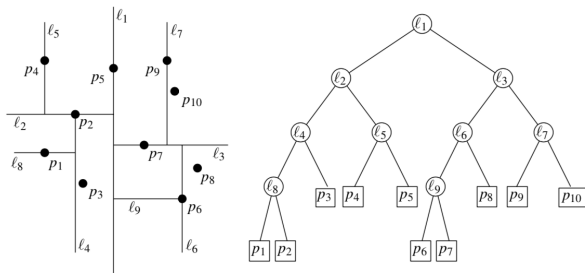


Figura extraída de Computational Geometry, de Berg et al., 2008

Kd-tree

Construção

- ▶ Partição à esquerda é intervalo fechado à direita
 - ▶ Ponto sobre plano divisor armazenado à esquerda
 - ▶ Assume, a princípio que não existem coordenadas x ou y coincidentes



Recursos

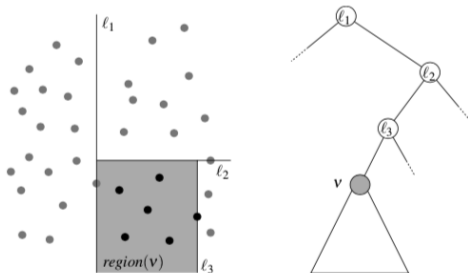
- ▶ Memória: $O(n)$
- ▶ Tempo de construção: $O(n \log n)$



Kd-tree

Propriedade

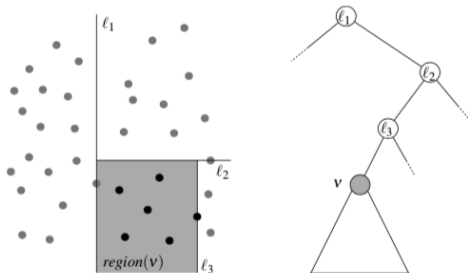
- Cada nó representa uma região (limitada ou não) por planos ancestrais



Kd-tree

Propriedade

- Cada nó representa uma região (limitada ou não) por planos ancestrais



- Algoritmo de busca
 - Nó só é visitado se sua região intercepta a região de interesse



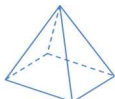
Cell-tree

Estrutura para localização de pontos em malhas não estruturadas

- *Fast, Memory-Efficient Cell Location in Unstructured Grids for Visualization*
- Garth e Joy, 2010



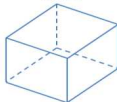
Tetrahedron



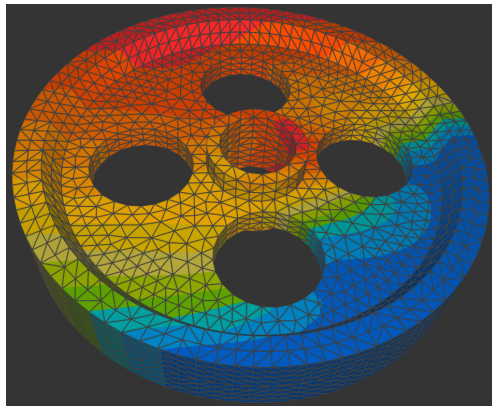
Pyramid



Triangular Prism



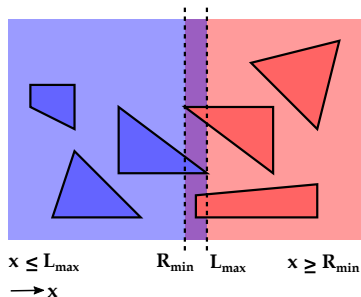
Hexahedron



Cell-tree

Hierarquia de intervalos envolventes

- ▶ Árvore binária de caixas alinhadas aos eixos (AABBs)
- ▶ Nós com sobreposição para não subdividir elementos
 - ▶ Partição disjunta da lista de elementos
 - ▶ Nó armazena dois limites: L_{max} e R_{min}
- ▶ Busca pode percorrer os dois filhos



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo da busca



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo da busca
- ▶ Custo da busca no nó i
 - ▶ Proporcional à propabilidade, ao número de filhos e ao custo da travessia

$$C_i = P(L)N_L + P(R)N_R + C_{trav}$$



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo da busca
- ▶ Custo da busca no nó i
 - ▶ Proporcional à propabilidade, ao número de filhos e ao custo da travessia

$$C_i = P(L)N_L + P(R)N_R + C_{trav}$$

- ▶ Assumindo consultas uniformemente distribuídas no espaço
 - ▶ Propabilidade dada pelo volume

$$C_i = V(L)N_L + V(R)N_R + C_{trav}$$



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo da busca
- ▶ Custo da busca no nó i
 - ▶ Proporcional à probabilidade, ao número de filhos e ao custo da travessia

$$C_i = P(L)N_L + P(R)N_R + C_{trav}$$

- ▶ Assumindo consultas uniformemente distribuídas no espaço
 - ▶ Probabilidade dada pelo volume

$$C_i = V(L)N_L + V(R)N_R + C_{trav}$$

- ▶ Objetivo: minimizar custo local

$$C_i = V(L)N_L + V(R)N_R$$



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo local

$$C_i = V(L)N_L + V(R)N_R$$



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo local

$$C_i = V(L)N_L + V(R)N_R$$

Split-middle: partição no meio da AABB

- ▶ Balanceamento de volumes: $V(L)$ e $V(R)$
- ▶ Possível desbalanceamento de número de filhos: N_L e N_R



Cell-tree

Critério para construção da hierarquia

- ▶ Objetivo: minimizar custo local

$$C_i = V(L)N_L + V(R)N_R$$

Split-middle: partição no meio da AABB

- ▶ Balanceamento de volumes: $V(L)$ e $V(R)$
- ▶ Possível desbalanceamento de número de filhos: N_L e N_R

Split-median: partição na mediana do conjunto

- ▶ Balanceamento de número de filhos: N_L e N_R
- ▶ Possível desbalanceamento de volumes: $V(L)$ e $V(R)$



Campo de Distância



Ref: 3D Distance Fields: A Survey of Techniques and Applications, Jones et al., 2006

Campo de distância é uma representação onde, para cada ponto do campo, sabe-se sua distância em relação ao ponto mais próximo do objeto.

Aplicações:

Em geral, usado como técnica de aceleração.

- ▶ Modelagem
- ▶ Manipulação/edição
- ▶ Visualização
- ▶ Detecção de colisão



Campo de distância contínuo

Campo de distância sem sinal

Considerando o conjunto Σ :

$$\text{dist}_{\Sigma}(\mathbf{p}) = \inf_{\mathbf{x} \in \Sigma} \|\mathbf{x} - \mathbf{p}\|$$

Campo de distância com sinal

Considerando o sólido S , a função de distância com sinal retorna a distância à fronteira ∂S :

$$d_S(\mathbf{p}) = \text{sign}(\mathbf{p}) \inf_{\mathbf{x} \in \partial S} \|\mathbf{x} - \mathbf{p}\|$$

$$\text{sign}(\mathbf{p}) = \begin{cases} -1 & \text{if } \mathbf{p} \in S \\ 1 & \text{otherwise} \end{cases}$$



Gradiente do campo de distância

Iso-superfície

É um conjunto de pontos: $\mathbf{p} / d(\mathbf{p}) = \tau$, onde τ é o iso-valor.

Gradiente

Para campo de distância com sinal, tem-se a seguinte propriedade:

$$\|\nabla d\| = 1$$

- ▶ Com exceção em pontos equidistantes a múltiplos pontos da superfície (e.g. centro de uma esfera)
 - ▶ Nestes pontos, o gradiente não é definido
 - ▶ Nos demais pontos, o gradiente é ortogonal à iso-superfície que passa no ponto



Derivada de segunda ordem

Provê informações sobre a curvatura das iso-superfícies.

Matrix de Hessian:

$$H = \begin{pmatrix} d_{xx} & d_{xy} & d_{xz} \\ d_{yx} & d_{yy} & d_{yz} \\ d_{zx} & d_{zy} & d_{zz} \end{pmatrix}$$

- ▶ Curvatura média:

$$k_M = \frac{1}{2}(d_{xx} + d_{yy} + d_{zz})$$

- ▶ Curvatura Gaussiana:

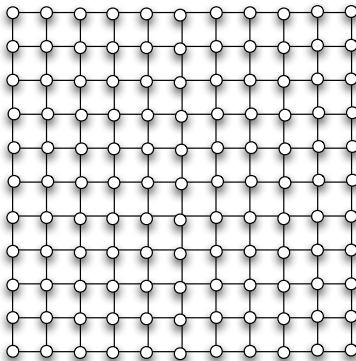
$$k_G = \begin{vmatrix} d_{xx} & d_{xy} \\ d_{yx} & d_{yy} \end{vmatrix} + \begin{vmatrix} d_{xx} & d_{xz} \\ d_{zx} & d_{zz} \end{vmatrix} + \begin{vmatrix} d_{yy} & d_{yz} \\ d_{zy} & d_{zz} \end{vmatrix}$$

- ▶ As curvaturas principais são dadas pelos dois auto-vetores diferentes de zero de H
 - ▶ O outro auto-valor é zero, significando que d varia linearmente ao longo do gradiente



Campo de distância discreto

Obtido por uma amostragem de um campo de distância contínuo.



Reconstrução

Em geral, estimada através de núcleos de reconstrução finitos.



Construção de campos de distância

Algoritmo força bruta:

- ▶ Para cada voxel (amostra), calcula a menor distância ao objeto
- ▶ Na prática, inviável devido ao alto custo computacional

Objetos representados por malhas de triângulos:

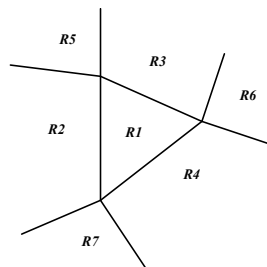
- ▶ Malhas devem ser fechadas, orientáveis e *2-manifold*
 - ▶ Malha não contém auto-interseção
 - ▶ Cada aresta tem exatamente dois triângulos adjacentes
 - ▶ Triângulos adjacentes a um vértice devem formar um único ciclo ao redor do vértice
- ▶ Algoritmo força-bruta: $O(N.M)$



Campo de distância para malhas de triângulos

Distância entre ponto e triângulo:

- ▶ Projeção do ponto no plano do triângulo
 - ▶ Região $R1$
 - ▶ Distância do ponto ao plano
 - ▶ Regiões $R2, R3, R4$
 - ▶ Distância do ponto à reta correspondente
 - ▶ Regiões $R5, R6, R7$
 - ▶ Distância do ponto ao ponto (vértice) correspondente



Campo de distância para malhas de triângulos

Técnicas de aceleração

- ▶ Reduzir triângulos testados explorando coerência espacial
 - ▶ Uso de hierarquias: $O(N.M) \rightarrow O(N.\log(M))$
- ▶ Calcular apenas distâncias de voxel próximos e propagar para demais voxels
 - ▶ Uso de transformadas de distância



Uso de hierarquia

Hierarquia de caixas envolventes (AABB)

- ▶ Para cada triângulo, calcula a caixa envolvente
- ▶ Triângulos são agrupados por coerência espacial, formando uma hierarquia de caixas envolventes

Cálculo da distância a um ponto

- ▶ Para cada caixa visitada, calcula d_{min} e d_{max}
- ▶ O valor d_{min} é usado como chave de uma lista de prioridade
- ▶ Caixas com d_{min} maior que um d_{max} de outra caixa, podem ser descartadas



Distância de voxels próximos à superfície

Inversão do processo:

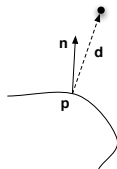
- ▶ Para cada triângulo, calcula sua AABB
- ▶ Calcula a distância para todos os voxels dentro da AABB
- ▶ Guarda o valor mínimo de cada voxel
- ▶ Propaga para demais voxels



Determinação do sinal

Superfície com continuidade C^1

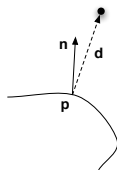
$$\mathbf{n} \cdot \mathbf{d} = \begin{cases} > 0 & \text{outside} \\ < 0 & \text{inside} \end{cases}$$



Determinação do sinal

Superfície com continuidade C^1

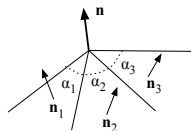
$$\mathbf{n} \cdot \mathbf{d} = \begin{cases} > 0 & \text{outside} \\ < 0 & \text{inside} \end{cases}$$



Malhas de triângulos não são C^1

- ▶ Determinação com traçado de raio
- ▶ Uso de pseudo-normais
 - ▶ Normal de vértices
 - ▶ Média ponderada das normais dos triângulos incidentes

$$\mathbf{n} = \frac{\sum_i \mathbf{n}_i \alpha_i}{\|\sum_i \mathbf{n}_i \alpha_i\|}$$



- ▶ Normal de arestas
 - ▶ Extensão da mesma ideia: ângulos iguais a π



Transformada de distância

Chamfer distance transformation

Forward pass

	f		f			f	e	d	e	f
f	e	d	e	f		e	c	b	c	e
	d		d			d	b	a	b	d
f	e	d	e	f		e	c	b	c	e
	f		f			f	e	d	e	f

z=-2

z=-1

	d		d		
d	b	a	b	d	
	a	0			

z=0

Backward pass

						f	e	d	e	f
						e	c	b	c	e
		0	a			d	b	a	b	d
d	b	a	b	d		e	c	b	c	e
	d		d			f	e	d	e	f

z=0

z=1

	f		f		
f	e	d	e	f	
	d		d		
f	e	d	e	f	
	f		f		

z=2

Transform	a	b	c	d	e	f
City Block (Manhattan)	1					
Chessboard	1	1				
Quasi-Euclidean $3 \times 3 \times 3$	1	$\sqrt{2}$				
Complete Euclidean $3 \times 3 \times 3$	1	$\sqrt{2}$	$\sqrt{3}$			
$\langle a, b, c \rangle_{opt} 3 \times 3 \times 3$ [SB02]	0.92644	1.34065	1.65849			
Quasi-Euclidean $5 \times 5 \times 5$	1	$\sqrt{2}$	$\sqrt{3}$	$\sqrt{5}$	$\sqrt{6}$	3

```

/* Forward Pass */
FOR(z = 0; z < f_z; z++)
  FOR(y = 0; y < f_y; y++)
    FOR(x = 0; x < f_x; x++)
      F[x,y,z] =
        inf_{i,j,k \in f_p} (F[x+i,y+j,z+k] + m[i,j,k])

```

```

/* Backward Pass */
FOR(z = f_z-1; z >= 0; z--)
  FOR(y = f_y-1; y >= 0; y--)
    FOR(x = f_x-1; x >= 0; x--)
      F[x,y,z] =
        inf_{i,j,k \in b_p} (F[x+i,y+j,z+k] + m[i,j,k])

```



Transformada de distância

Chamfer distance transformation



Aritmética Robusta



Aritmética Robusta

Representação de ponto flutuante

$$\pm 1.bbbb... \times 2^{eee...}$$

Tipos ponto flutuante no computador

► Número de bits

	sinal	mantissa	expoente
float	1	23	8
double	1	52	11



Aritmética Robusta

Problema

$$a + b + c \neq a + c + b$$



Aritmética Robusta

Problema

$$a + b + c \neq a + c + b$$

► Exemplo

```
printf("%.16g\n", 1.2 - 1.0 - 0.2);  -- saída: -5.5511151231258e-17  
printf("%.16g\n", 1.2 - 0.2 - 1.0);  -- saída: 0.0
```



Aritmética Robusta

Problema

$$a + b + c \neq a + c + b$$

► Exemplo

```
printf("%.16g\n", 1.2 - 1.0 - 0.2);  -- saída: -5.5511151231258e-17
printf("%.16g\n", 1.2 - 0.2 - 1.0);  -- saída: 0.0
```

► Razão: 1.2 e 0.2 não têm representação finita na base binária

$$(1.2)_{10} = (1.\overline{0011})_2$$

$$(0.2)_{10} = (0.\overline{0011})_2 = (1.\overline{1001})_2 \times 2^{-3}$$

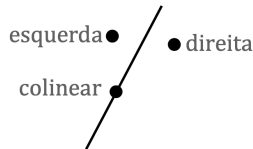


Aritmética Robusta

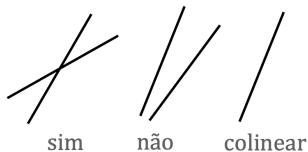
Predicados geométricos

- ▶ Testam propriedades de objetos
- ▶ Usados em decisões condicionais: determinam fluxo do algoritmo
- ▶ Resultados incorretos ou contraditórios podem levar a estado inválido

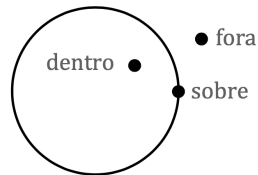
Ponto em relação a reta



Interseção de segmentos



Ponto em relação a círculo

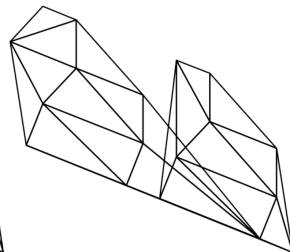
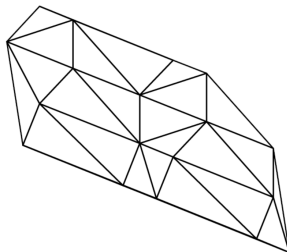
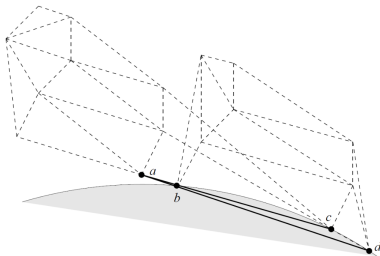


Aritmética Robusta

Falha em algoritmos geométricos

- ▶ Triangulação de Delaunay

- ▶ Localização do ponto b em relação às retas $\overline{ac}$ e $\overline{ad}$
- ▶ Localização de ponto a em relação a círculo dcb



Shewchuk, J. R., "Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates", 1997.



Aritmética Robusta

Representação binária finita

$$r = \pm 1.bbbb... \times 2^{eee...} = \pm 1.\mathbf{b} \times 2^{\mathbf{e}}$$

- onde $\mathbf{b}$ e $\mathbf{e}$ são vetores finitos, mas dinâmicos, de bits



Aritmética Robusta

Representação binária finita

$$r = \pm 1.bbbb... \times 2^{eee...} = \pm 1.\mathbf{b} \times 2^{\mathbf{e}}$$

- ▶ onde $\mathbf{b}$ e $\mathbf{e}$ são vetores finitos, mas dinâmicos, de bits

Operações aritméticas robustas

- ▶ Adição, subtração, multiplicação
 - ▶ Elevado custo computacional com realocação de $\mathbf{b}$ e $\mathbf{e}$



Aritmética Robusta

Número racionais: x/y

- ▶ x e y com representações finitas

Suporte às quatro operações aritméticas

- ▶ Adição e subtração

$$r_1 \pm r_2 = \frac{x_1}{y_1} \pm \frac{x_2}{y_2} = \frac{x_1 y_2 \pm x_2 y_1}{y_1 y_2}$$



Aritmética Robusta

Número racionais: x/y

- ▶ x e y com representações finitas

Suporte às quatro operações aritméticas

- ▶ Adição e subtração

$$r_1 \pm r_2 = \frac{x_1}{y_1} \pm \frac{x_2}{y_2} = \frac{x_1 y_2 \pm x_2 y_1}{y_1 y_2}$$

- ▶ Multiplicação

$$r_1 r_2 = \frac{x_1}{y_1} \frac{x_2}{y_2} = \frac{x_1 x_2}{y_1 y_2}$$



Aritmética Robusta

Número racionais: x/y

- Suporte a divisão

$$\frac{r_1}{r_2} = \frac{\frac{x_1}{y_1}}{\frac{x_2}{y_2}} = \frac{x_1 y_2}{x_2 y_1}$$



Aritmética Robusta

Número racionais: x/y

- Suporte a divisão

$$\frac{r_1}{r_2} = \frac{\frac{x_1}{y_1}}{\frac{x_2}{y_2}} = \frac{x_1 y_2}{x_2 y_1}$$

Representação binárias não finitas

- Converte para número racional: x/y

